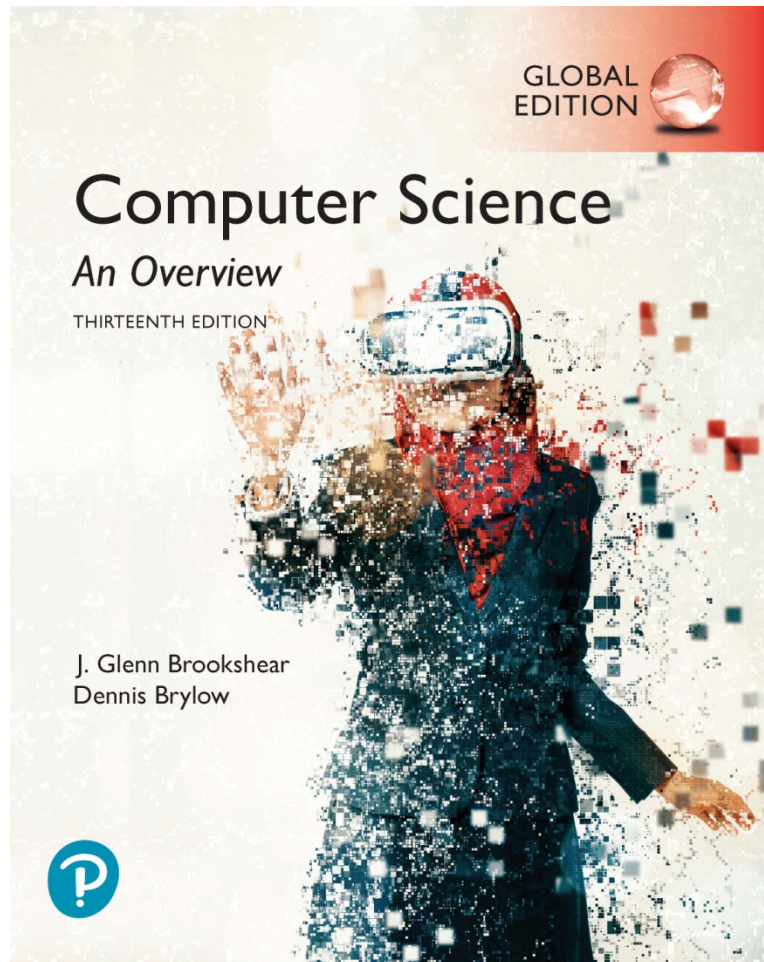


Computer Science An Overview

13th Edition, Global Edition



Chapter 2 Data Manipulation

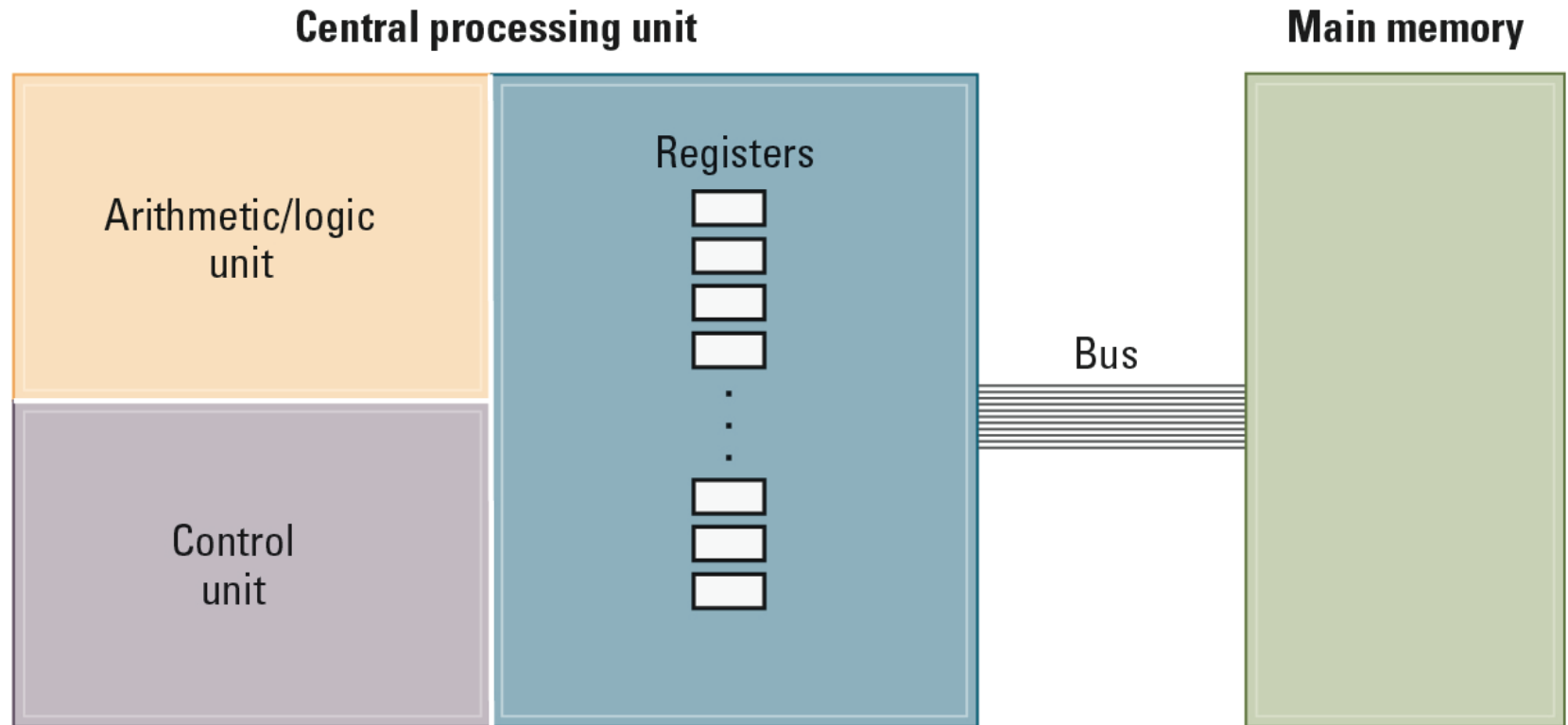
Chapter 2: Data Manipulation

- 2.1 Computer Architecture
- 2.2 Machine Language
- 2.3 Program Execution
- 2.4 Arithmetic/Logic
算術、計算
- 2.5 Communicating with Other Devices
- 2.6 Program Data Manipulation
- 2.7 Other Architectures

2.1 Computer Architecture

- Central Processing Unit (CPU)
 - Arithmetic/Logic Unit
 - Control Unit
 - Register Unit
 - General purpose registers
 - Special purpose registers
- Bus 匯流排
- Main Memory 主要記憶體

Figure 2.1 CPU and main memory connected via a bus



Stored Program Concept

A program can be encoded as bit patterns and stored in Main Memory. From there, the Control Unit can extract, (Fetch) decode, and execute instructions.

解碼

執行

取出(截取)

(main memory → Registers)

Instead of rewiring the CPU, the program can be altered by changing the contents of Main Memory.

2.2 Machine Language

機器指令

- **Machine instruction:** An instruction encoded as a bit pattern recognizable by the CPU
- **Machine language:** The set of all instructions recognized by a machine

機器語言

Machine Language Philosophies

- Reduced Instruction Set Computing (RISC) 精簡指令集
 - Few, simple, efficient, and fast instructions
 - Examples: PowerPC from Apple/IBM/Motorola and ARM
- Complex Instruction Set Computing (CISC) (複雜指令集)
 - Many, convenient, and powerful instructions
 - Example: Intel

Machine Instruction Types

資料轉移

- Data Transfer: **copy data** from one location **to another** (e.g. LOAD, STORE)
- 算術邏輯

Arithmetic/Logic: operations on bit patterns (e.g. +, -, *, /, AND, OR, SHIFT, ROTATE)
- 控制執行

Control: direct the execution of the program (e.g. JUMP, BRANCH)

*詳見附錄C, P. 675

Figure 2.2 Adding values stored in memory

Step 1. Get one of the values to be added from memory and place it in a register.

Step 2. Get the other value to be added from memory and place it in another register.

將兩數從 Memory 上
移轉至 Register

Step 3. Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.

啟動加法电路，
將兩數相加並將
結果寫回暫存器

Step 4. Store the result in memory. → 將結果從暫存器
寫回記憶體

Step 5. Stop.

Figure 2.3 Dividing values stored in memory

Step 1. **LOAD** a register with a value from memory.

Step 2. **LOAD** another register with another value from memory.

將兩數之值
從 Memory 移轉至
Registers

Step 3. If this second value is zero, **JUMP** to Step 6.

⇒ 判斷除數是否為 0

Step 4. Divide the contents of the first register by the second register and leave the result in a third register.

將兩數相除
並將結果存於
暫存器

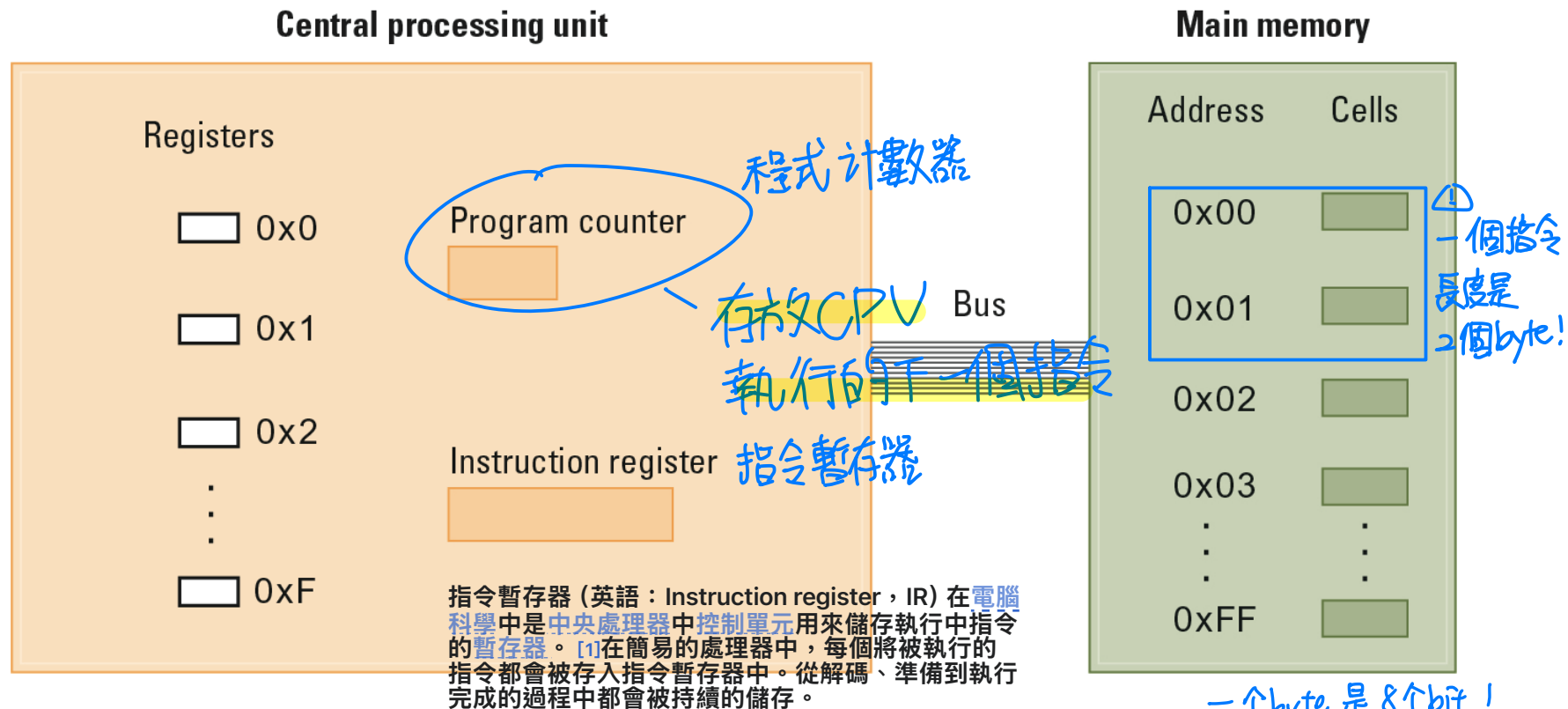
Step 5. **STORE** the contents of the third register in memory.

→ 將結果從暫存器

Step 6. **STOP.**

寫回記憶體

Figure 2.4 The architecture of the Vole, as described in Appendix C



程式計數器

存放CPU
執行的下一個指令

指令暫存器

一個指令
長度是
2個byte!

一個byte是8個bit!

* P.16 所以一個指令會需要暫用
2個byte (16個bit)

Parts of a Machine Instruction

- **Op-code:** ^{操作碼(控制碼)} Specifies which operation to execute ^{, 指定要執行的操作}
- **Operand:** ^{運算元} Gives more detailed information about the operation → ^{負責提供操作的資料, 如 Register 位置、Memory 位置、算術邏輯等……}
- Interpretation of operand varies depending on op-code

Figure 2.5 The composition of a Vole instruction

Review: 一個 Memory cell 有一個 1 個 byte (8 bit)

<key: 一個指令要占用 兩個 Memory cell />
16 個 bit 、 2 byte

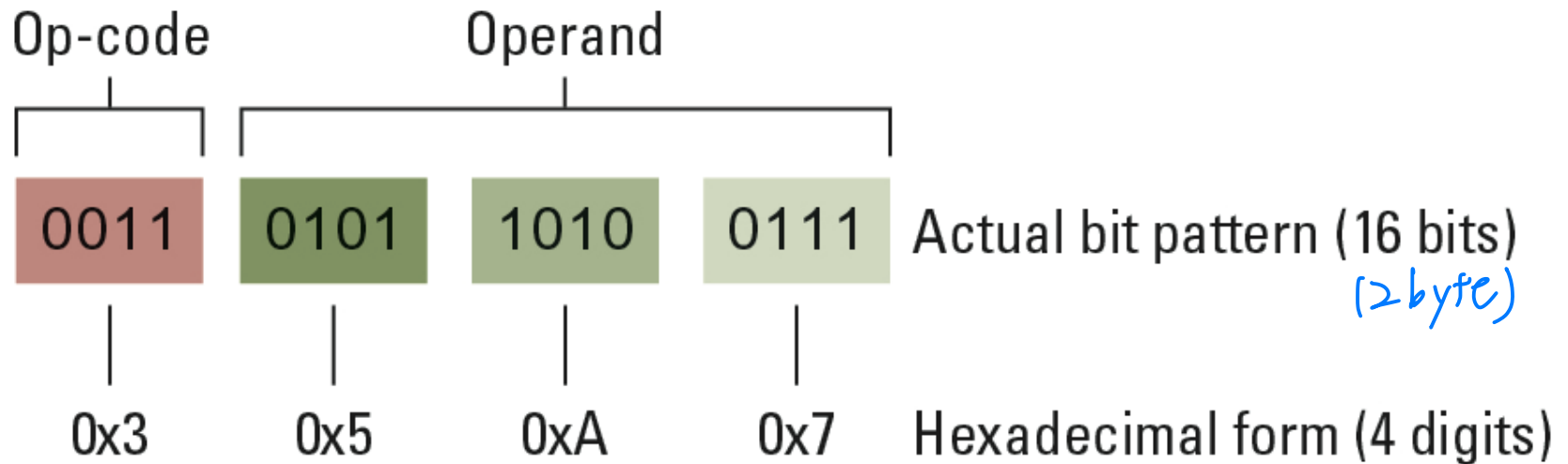


Figure 2.6 Decoding the instruction 0x35A7

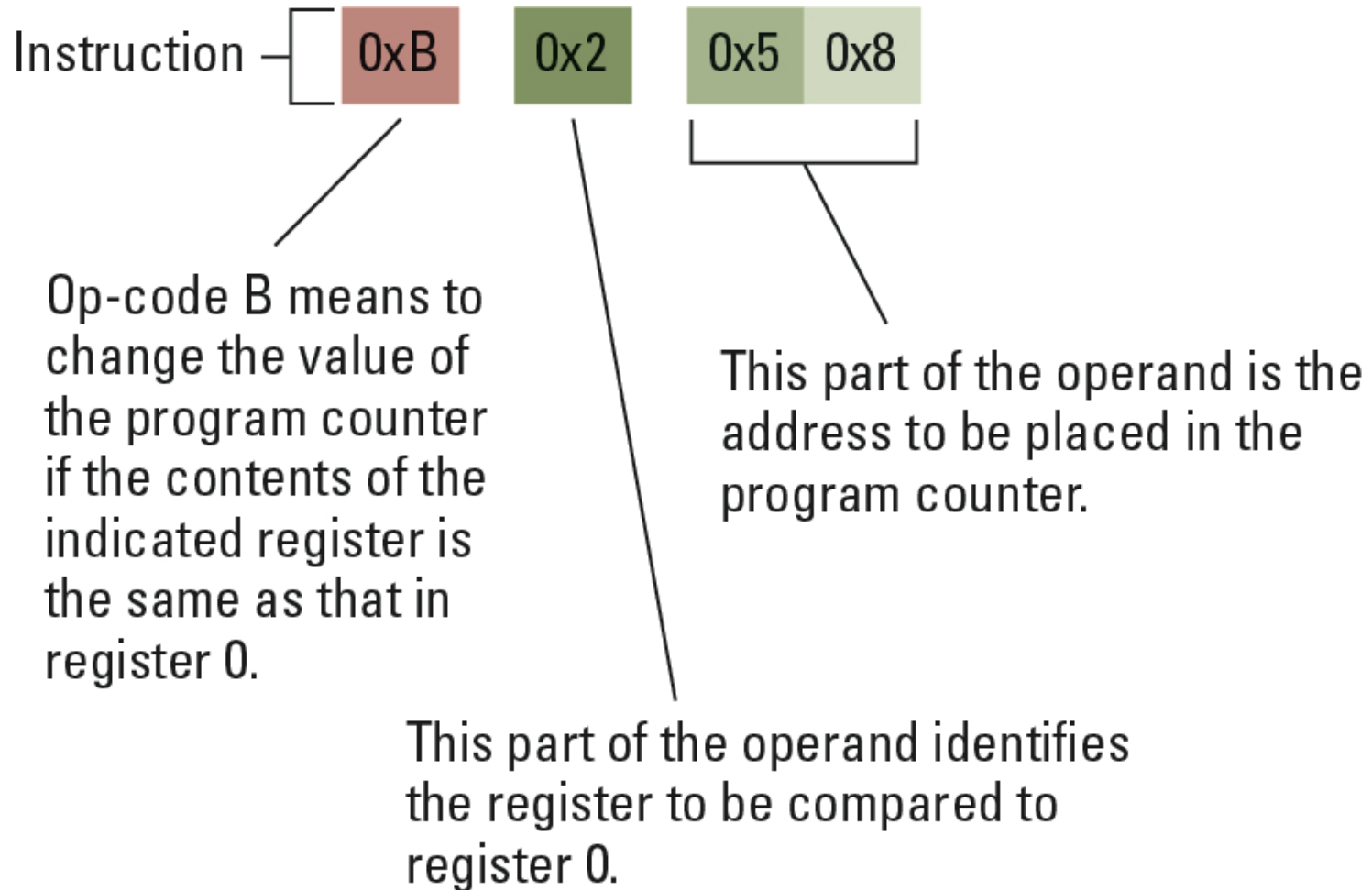


Figure 2.7 An encoded version of the instructions in Figure 2.2

Encoded instructions	Translation
0x156C	Load register 0x5 with the bit pattern found in the memory cell at address 0x6C.
0x166D	Load register 0x6 with the bit pattern found in the memory cell at address 0x6D.
0x5056	Add the contents of register 0x5 and 0x6 as though they were two's complement representation and leave the result in register 0x0.
0x306E	Store the contents of register 0x0 in the memory cell at address 0x6E.
0xC000	Halt.

2.3 Program Execution

- Controlled by two special purpose registers
 - Instruction register 通用暫存器
 - holds current instruction → 儲存目前的指令
 - Program counter 程式計數器
 - holds address of next instruction
 - 、儲存下一條指令的位置
 - Machine Cycle: (repeat these 3 steps)
 - Fetch, Decode, Execute
 - 擷取 解碼 執行
- <Link = P.11>

Figure 2.8 The machine cycle

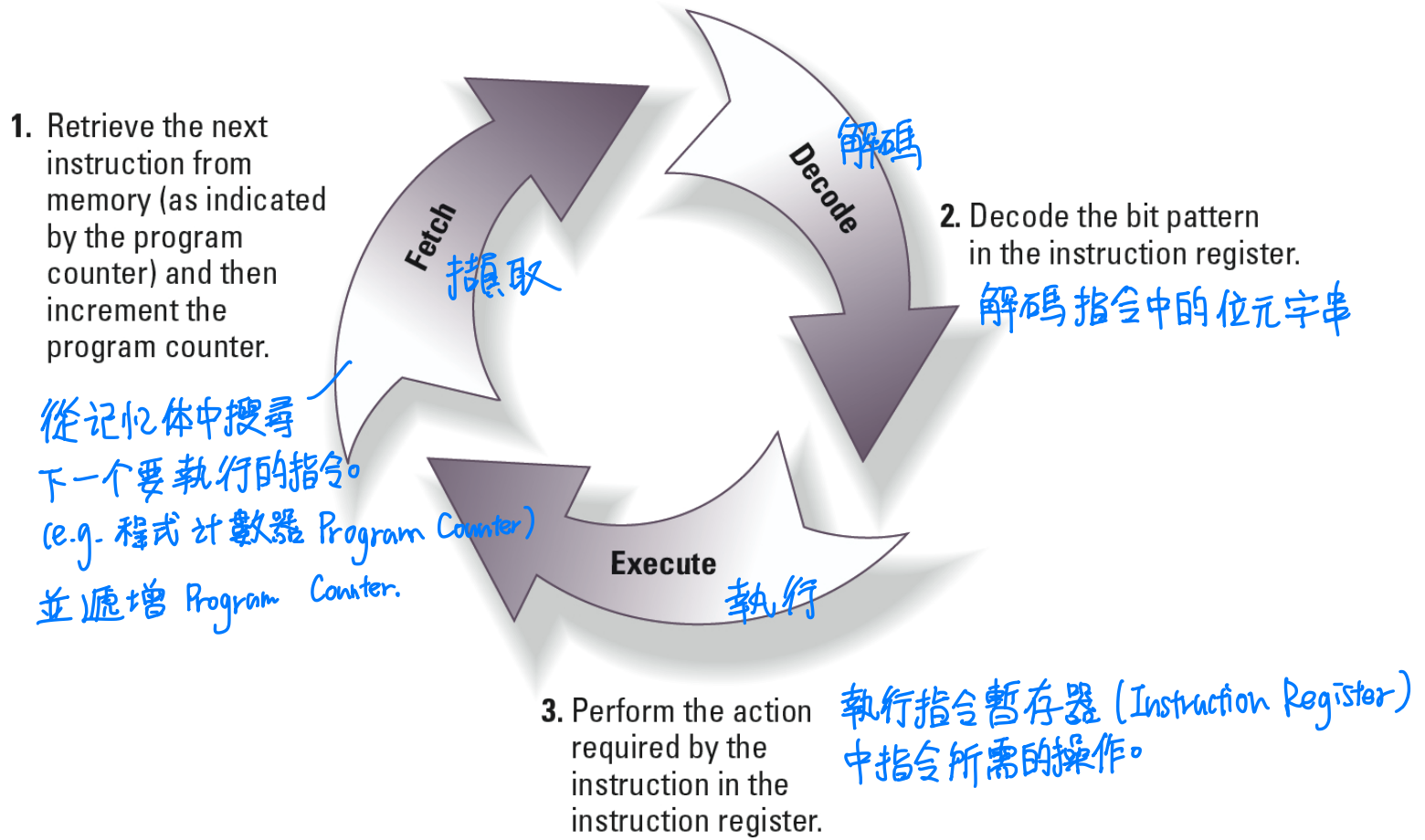


Figure 2.9 Decoding the instruction B258

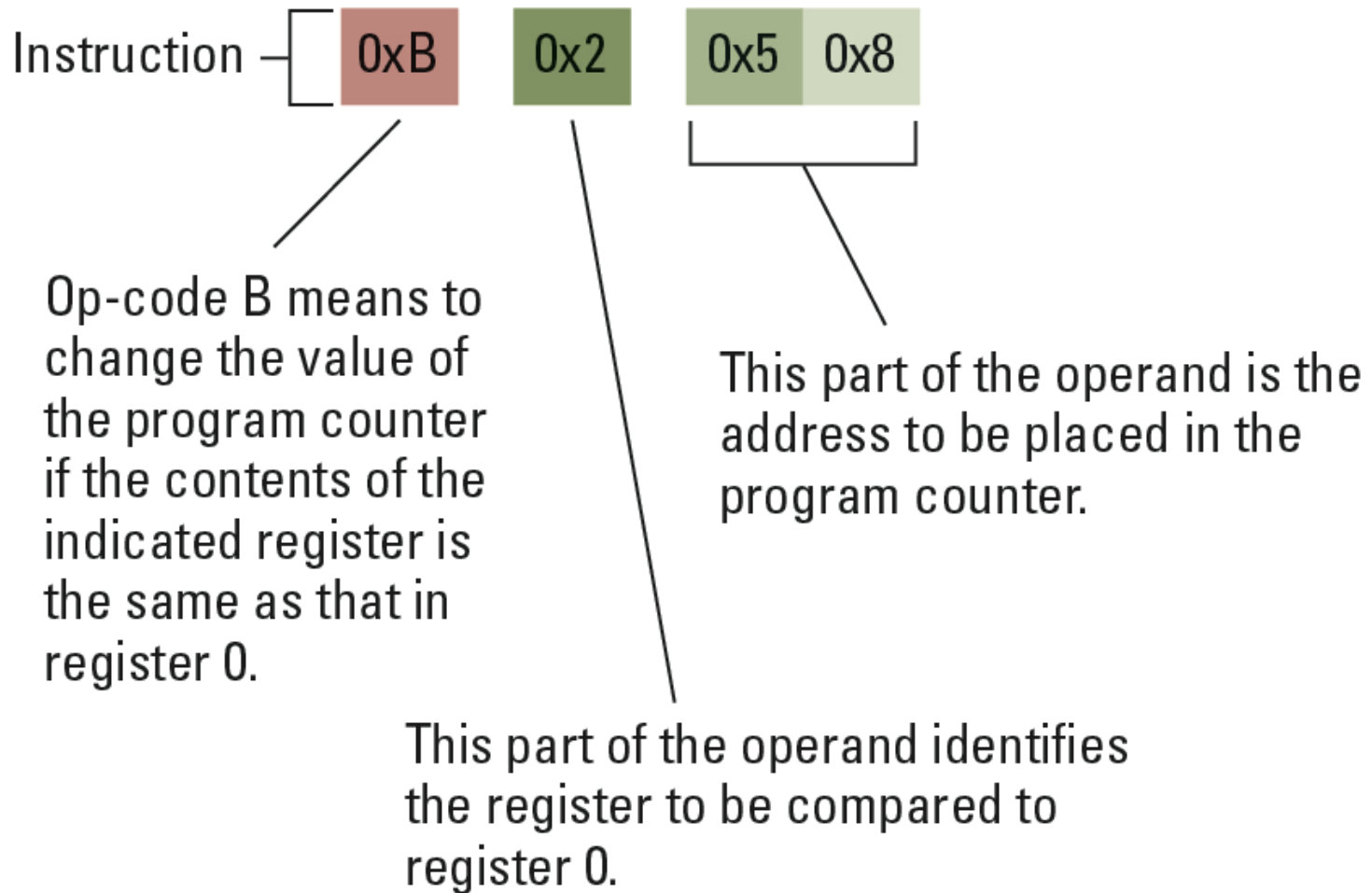


Figure 2.10 The program from Figure 2.7 stored in main memory ready for execution

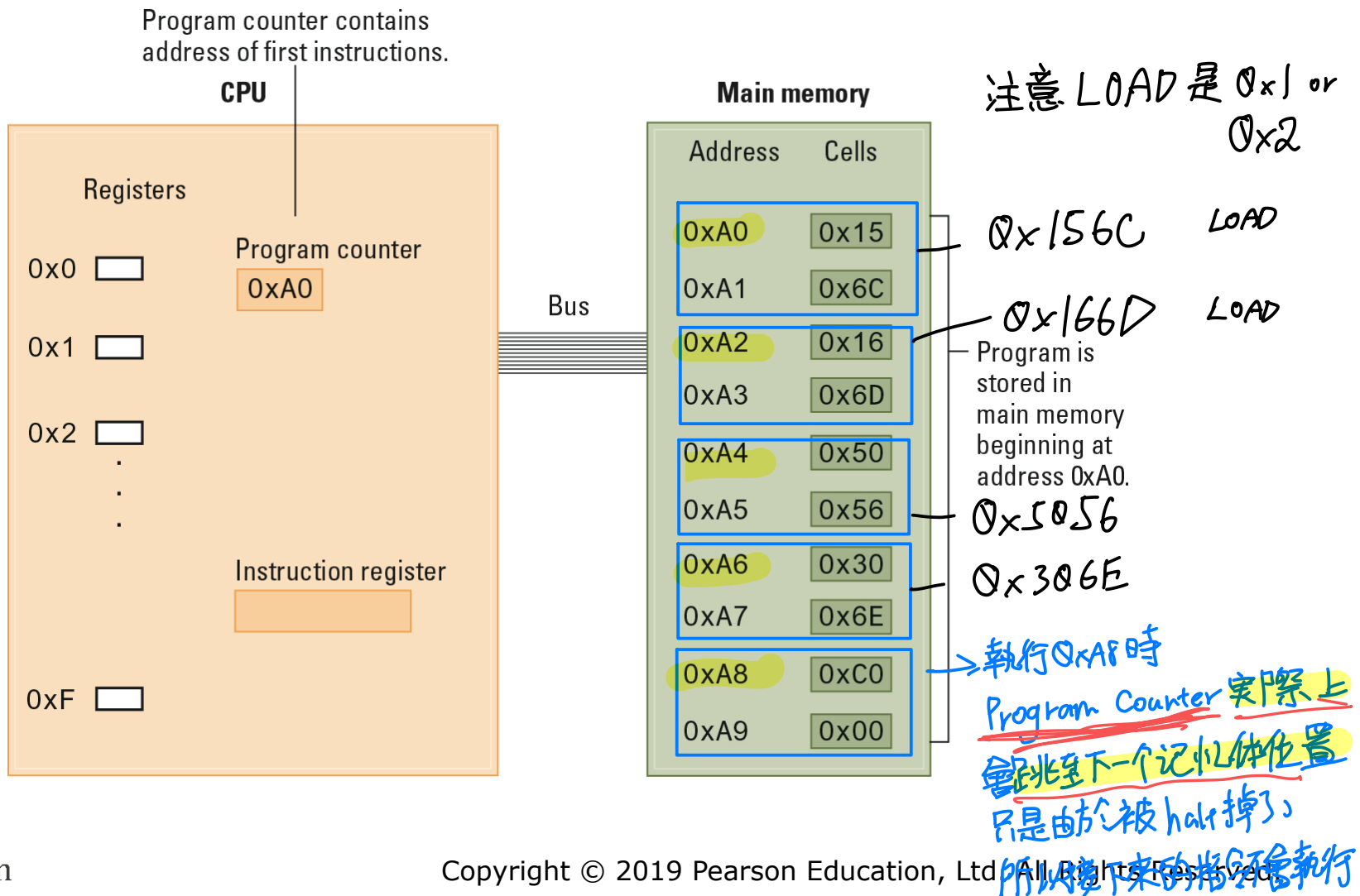
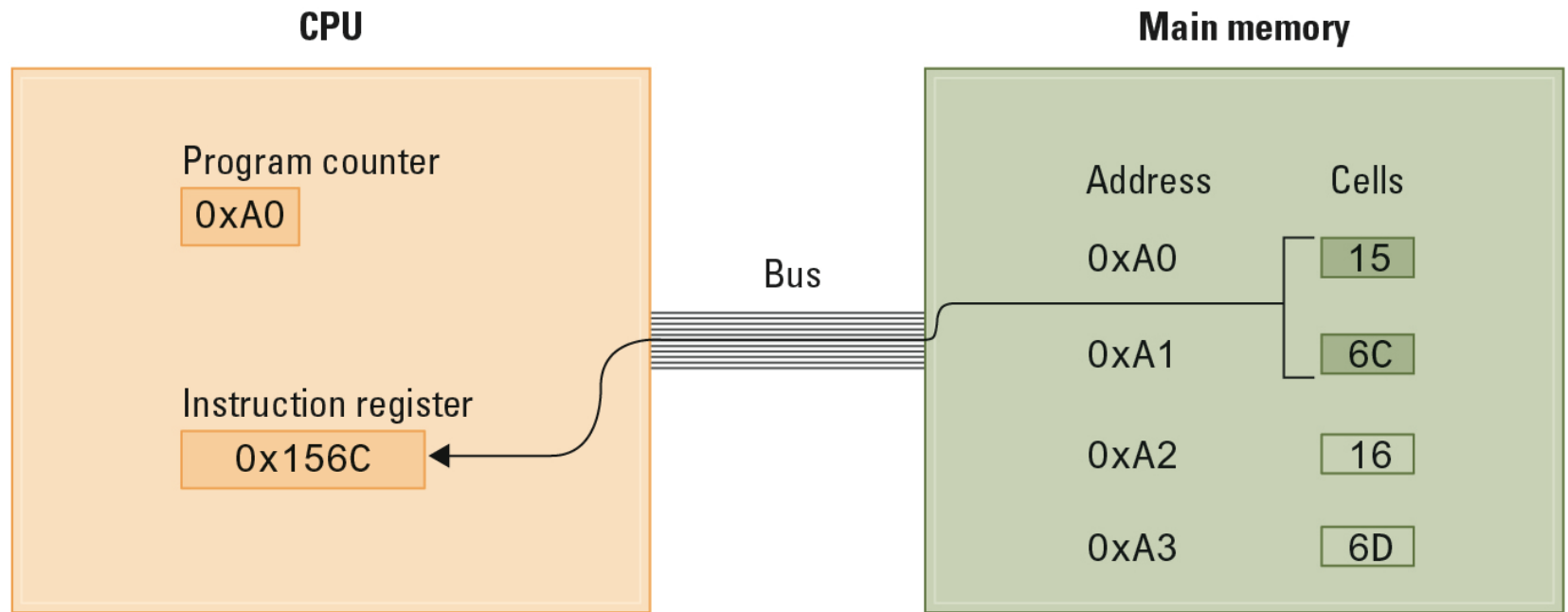


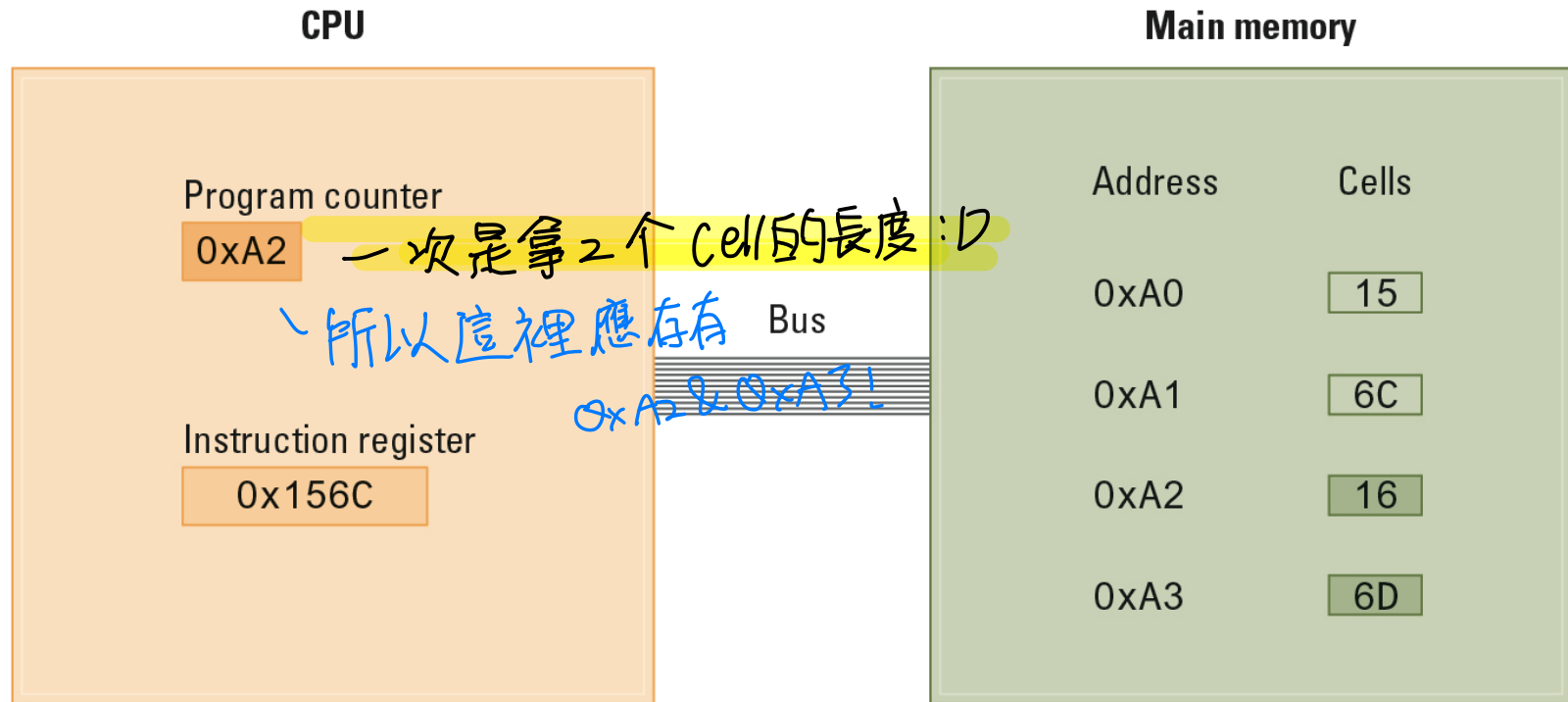
Figure 2.11 Performing the fetch step of the machine cycle



- a. At the beginning of the fetch step, the instruction starting at address 0xA0 is retrieved from memory and placed in the instruction register.

在一開始擷取時，會先從記憶體 0xA0 的地址開始擷取，並將其放入 Instruction Register (指令暫存器)

Figure 2.11 Performing the fetch step of the machine cycle (continued)



b. Then the program counter is incremented so that it points to the next instruction.

然後程式計數器 遞增, 指向下一個指令。

算術

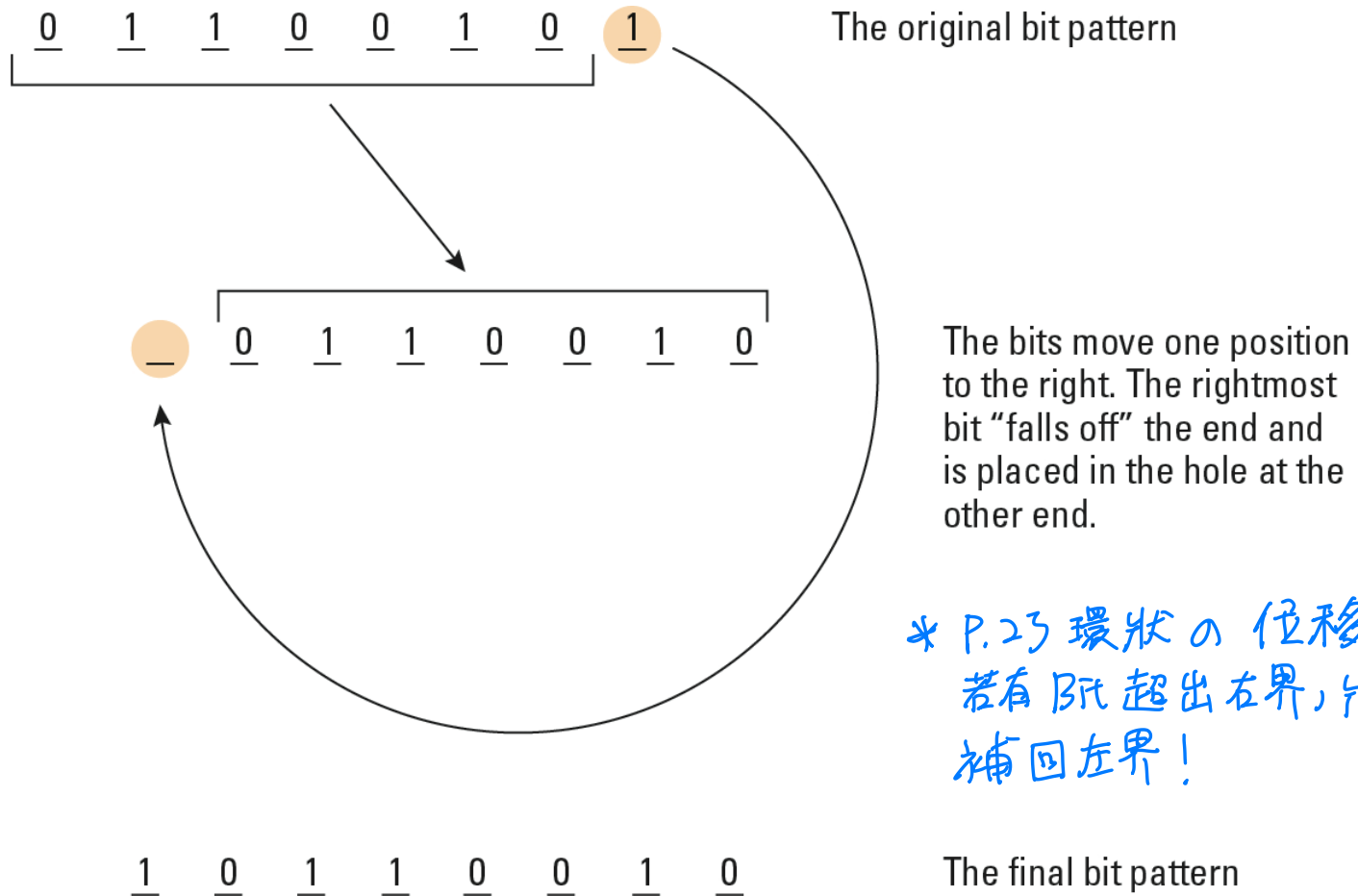
邏輯

2.4 Arithmetic/Logic Instructions

*P.131

- Logic Operations:
 - AND, OR, XOR
 - often used to mask an operand
- Rotation and Shift Operations: 位移
 - circular shift, logical shift, arithmetic shift
 - 超出的部分一率補0捨棄)
環狀的shift, 若有bit超出右界將補回左界 算術位移
- Arithmetic Operations:
 - add, subtract, multiply, divide
 - two's complement versus floating-point
 - 二補數 & 浮點數系統

Figure 2.12 Rotating the bit pattern 0x65 one bit to the right



2.5 Communicating with Other Devices

控制器

- **Controller:** handles communication between the computer and other devices 處理電腦和其他設備間的通信

- Specialized (by type of device)

- General purpose (USB, HDMI) 通用控制器

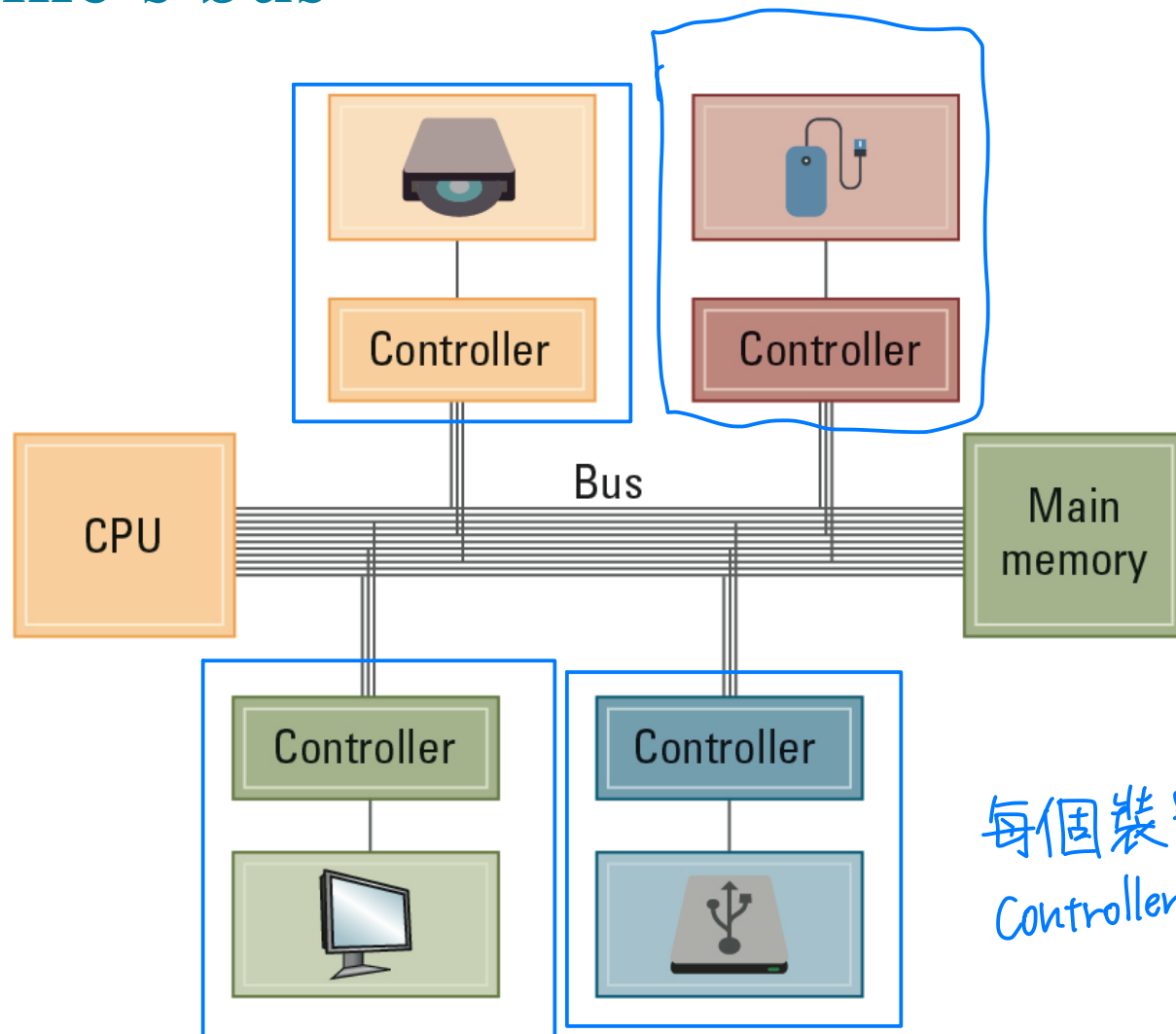
連接埠(端口)

- **Port:** The point at which a device connects to a computer

MMIO 記憶體對應輸入輸出

- **Memory-mapped I/O:** devices appear to the CPU as though they were memory locations

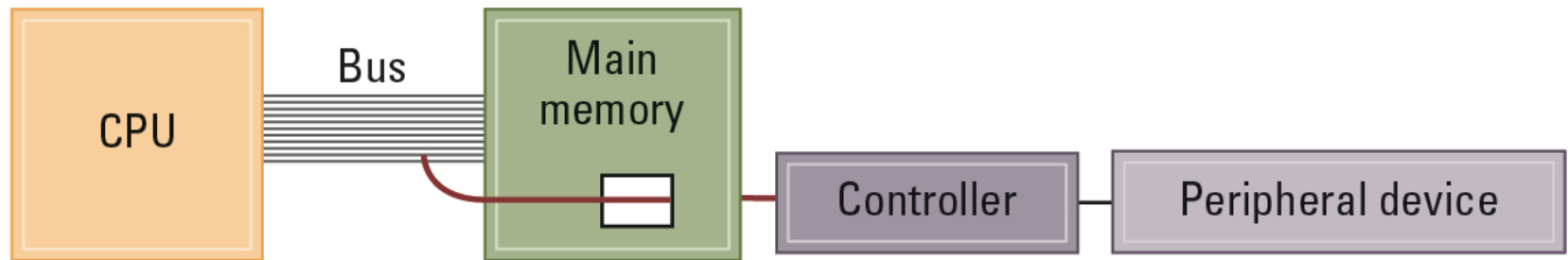
Figure 2.13 Controllers attached to a machine's bus



每個裝置都有一個
Controller，都在裝置內部

Figure 2.14 A conceptual representation of memory-mapped I/O

记忆体对应输入输出



Communicating with Other Devices (continued)

直接記憶體存取 (不需透過 CPU)

- **Direct memory access (DMA):** Main memory access by a controller over the bus
(效能瓶頸) 控制器可透過匯流排存取主記憶體
- **Von Neumann Bottleneck:** occurs when the CPU and controllers compete for bus access
(馮紐曼瓶頸) 當CPU和控制器競爭總線的訪問時,會發生馮紐曼瓶頸
- 握手 **Handshaking:** the process of coordinating the transfer of data between the computer and the peripheral device
協調電腦及周邊設備之間資料傳輸的過程。

Communicating with Other Devices (continued)

- **Popular Communication Media**

並列通訊 (平行通訊)

多訊号同時傳輸

/ 每個訊号都在「單獨線路」

- **Parallel Communication: Several signals**

transferred at the same time, each on a separate “line” (computer’s internal bus)

序列通訊

- **Serial Communication: Signals are transferred one after the other over a single “line” (USB, FireWire)**

\ 訊号通過「一條線路」, 「逐一傳輸」

Data Communication Rates

- Measurement units
 - bps: bits per second
bit
 - Kbps: Kilo-bps (1,000 bps) $\times 10^3$
 - Mbps: Mega-bps (1,000,000 bps) $\times 10^3$
 - Gbps: Giga-bps (1,000,000,000 bps) $\times 10^3$
- Bandwidth: Maximum available rate
頻寬

2.6 Programming Data Manipulation

- Programing languages shields users from details of the machine:
 - A single Python statement might map to one, tens, or hundreds of machine instructions
 - Programmer does not need to know if the processor is RISC or CISC
 - Assigning variables surely involves LOAD, STORE, and MOVE op-codes
变数的宣告, 基本上就包含 LOAD、STORE & MOVE 等指令

Bitwise Problems as Python Code

^{二進位}
`print(bin(0b10011010 & 0b11001001))`
Prints '0b10001000' ^{AND 運算}

`print(bin(0b10011010 | 0b11001001))`
Prints '0b11011011' ^{OR 運算}
^{若沒有bin(), 將以十進位輸出!}

[↙]
`print(bin(0b10011010 ^ 0b11001001))`
Prints '0b1010011' ^{XOR 運算}

Control Structures

- If statement:

```
if (water_temp > 140):  
    print('Bath water too hot!')
```

- While statement:

```
while (n < 10):  
    print(n)  
    n = n + 1
```


Functions

函數的定義

應對給定參數執行的一系列操作的名稱。

- **Function:** A name for a series of operations that should be performed on the given parameter or parameters
- **Function call:** Appearance of a function in an expression or statement

x = 1034

y = 1056

z = 2078

Parameter (參數)

biggest = max(x, y, z)

print(biggest) # Prints '2078'

Functions (continued)

- **Argument Value:** A value plugged into a parameter 參數
- **Fruitful functions** **return** a value 有回傳值
- **void functions**, or **procedures**, do not return a value 無回傳值

sideA = 3.0

sideB = 4.0

Calculate third side via Pythagorean Theorem

hypotenuse = math.sqrt(sideA**2 + sideB**2)

print(hypotenuse)

Input / Output

Calculates the hypotenuse of a right triangle

```
import math
```

Inputting the side lengths, first try

```
sideA = int(input('Length of side A? '))
```

```
sideB = int(input('Length of side B? '))
```

Calculate third side via Pythagorean Theorem

```
hypotenuse = math.sqrt(sideA**2 + sideB**2)
```

```
print(hypotenuse)
```

Marathon Training Assistant

```
# Marathon training assistant.
```

```
import math
```

```
# This function converts a number of minutes and  
# seconds into just seconds.
```

```
def total_seconds(min, sec):  
    return min * 60 + sec
```

```
# This function calculates a speed in miles per hour given  
# a time (in seconds) to run a single mile.
```

```
def speed(time):  
    return 3600 / time
```

Marathon Training Assistant (continued)

```
# Prompt user for pace and mileage.
```

```
pace_minutes = int(input('Minutes per mile? '))
```

```
pace_seconds = int(input('Seconds per mile? '))
```

```
miles = int(input('Total miles? '))
```

```
# Calculate and print speed.
```

```
mph = speed(total_seconds(pace_minutes, pace_seconds))
```

```
print('Your speed is ' + str(mph) + ' mph')
```

```
# Calculate elapsed time for planned workout.
```

```
total = miles * total_seconds(pace_minutes, pace_seconds)
```

```
elapsed_minutes = total // 60
```

```
elapsed_seconds = total % 60
```

```
print('Your elapsed time is ' + str(elapsed_minutes) +  
      ' mins ' + str(elapsed_seconds) + ' secs')
```

Figure 2.15 Example Marathon Training Data

Time Per Mile				Total Elapsed Time	
Minutes	Seconds	Miles	Speed (mph)	Minutes	Seconds
9	14	5	6.49819494584	46	10
8	0	3	7.5	24	0
7	45	6	7.74193548387	46	30
7	25	1	8.08988764044	7	25

2.7 Other Architectures

- Technologies to increase throughput:
 - Pipelining: Overlap steps of the machine cycle
指令管線化 機器週期的步驟重疊
 - Parallel Processing: Use multiple processors simultaneously
平行運算
管線在處理器的內部被組織成層級，各個層級的管線能半獨立地單獨運作。每一個層級都被管理並且鏈結到一條「鏈」，因而每個層級的輸出被送到其它層級直至任務完成。處理器的這種組織方式能使總體的處理時間顯著縮短。
 - SISD: Single Instruction, Single Data
單指令流、單數據流
 - No parallel processing
沒有平行處理
 - MIMD: Multiple Instruction, Multiple Data
多指令流、多數據流
 - Different programs, different data
可以同時有不同程式、不同資料
 - SIMD: Single Instruction, Multiple Data
單指令流、多數據流。
 - Same program, different data
同個程式、平行處理多資料